Java 8 Stream API Interview Cheat Sheet

1. Common Stream Methods

filter() — Keep matching elements

map() — Transform elements

flatMap() — Flatten nested collections

distinct() — Remove duplicates

sorted() — Sort elements

limit() — Keep first N

skip() — Skip first N

collect() — Convert to List/Set/Map

count() — Count elements

findFirst() — First element

findAny() — Any element

max() — Largest element

min() — Smallest element

anyMatch() — Any matches

allMatch() — All match

noneMatch() — None match

mapToInt() — Convert to IntStream

sum() — Sum values

2. Common Collectors

Collectors.toList()

Collectors.toSet()

Collectors.toCollection(TreeSet::new)

Collectors.groupingBy()

Collectors.partitioningBy()

Collectors.joining()

Collectors.counting()

Collectors.toMap()

3. Stream Execution Flow

Collection → stream() → Intermediate Operations (filter, map, flatMap, distinct, sorted) → Terminal Operation (collect, count, findFirst)

Important Coding Examples

Remove Duplicates

```
nums.stream().distinct().toList();
```

Find Even Numbers

```
nums.stream().filter(n -> n % 2 == 0).toList();
```

Square Numbers
```
nums.stream().map(n -> n * n).toList();
```

Find Maximum
```
nums.stream().max(Integer::compare).get();
```

Sum of Numbers
```
nums.stream().mapToInt(Integer::intValue).sum();
```

Sort Descending
```
nums.stream().sorted(Collections.reverseOrder()).toList();
```

Top 3 Largest
```
nums.stream().sorted(Collections.reverseOrder()).limit(3).toList();
```

Join Strings
```
names.stream().collect(Collectors.joining(", "));
```

Flatten Comma-Separated Values
```
list.stream().flatMap(s -> Arrays.stream(s.split(","))).map(Integer::parseInt).toList();
```

Remove Duplicates & Sort
```
list.stream().flatMap(s -> Arrays.stream(s.split(","))).map(Integer::parseInt).distinct().sorted(Collecti
ons.reverseOrder()).toList();
```

Group Employees
```
employees.stream().collect(Collectors.groupingBy(Employee::getDepartment));
```

Count by Department
```
employees.stream().collect(Collectors.groupingBy(Employee::getDepartment,
Collectors.counting()));
```

Highest Salary
```
employees.stream().max(Comparator.comparing(Employee::getSalary)).get();
```

Second Highest
```
nums.stream().distinct().sorted(Collections.reverseOrder()).skip(1).findFirst().get();
```

Partition Even/Odd
```
nums.stream().collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

Any Number >100
```
nums.stream().anyMatch(n -> n > 100);
```

Additional Interview-Favorite Coding Questions

Find First Duplicate
```
nums.stream().filter(n -> Collections.frequency(nums, n) > 1).findFirst().orElse(-1);
```

Find Duplicate Elements
```
nums.stream().filter(n -> Collections.frequency(nums, n) > 1).distinct().toList();
```

Frequency of Each Character
```
str.chars().mapToObj(c -> (char)c).collect(Collectors.groupingBy(c -> c, Collectors.counting()));
```

Frequency of Each Word
```
Arrays.stream(sentence.split(" ")).collect(Collectors.groupingBy(w -> w, Collectors.counting()));
```

Longest String
```
names.stream().max(Comparator.comparing(String::length)).get();
```

Average of Numbers
```
nums.stream().mapToInt(Integer::intValue).average().getAsDouble();
```

Concatenate Two Lists
```
Stream.concat(list1.stream(), list2.stream()).toList();
```

Remove Null Values
```
names.stream().filter(Objects::nonNull).toList();
```

Find Common Elements

```
list1.stream().filter(list2::contains).distinct().toList();
```

First Non-Repeating Character

```
str.chars().mapToObj(c -> (char)c).filter(c ->
str.indexOf(c)==str.lastIndexOf(c)).findFirst().orElse(null);
```

Top 10 Stream Coding Problems (Most Asked in Interviews)

These problems commonly appear in Java backend interviews at EPAM, Nagarro, Deloitte, Cognizant, Capgemini, TCS Digital, and Proximity Works.

1. Flatten Nested Lists using flatMap

Input

```
List> nums = Arrays.asList(Arrays.asList(1,2), Arrays.asList(3,4));
```

Solution

```
nums.stream().flatMap(List::stream).toList();
```

2. Split Comma-Separated Strings

Input

```
List list = Arrays.asList("1,2","3,4","5,6");
```

Solution

```
list.stream().flatMap(s -> Arrays.stream(s.split(","))).toList();
```

3. Ignore Invalid Numbers using Regex

Input

```
List list = Arrays.asList("1","abc","5","10x","20");
```

Solution

```
list.stream().filter(s -> s.matches("\\d+")).map(Integer::parseInt).toList();
```

4. Remove Duplicates and Sort Descending

Input

```
List nums = Arrays.asList(5,2,5,8,1);
```

Solution

```
nums.stream().distinct().sorted(Collections.reverseOrder()).toList();
```

5. Second Highest Number

Input

```
List nums = Arrays.asList(10,40,20,40,30);
```

Solution

```
nums.stream().distinct().sorted(Collections.reverseOrder()).skip(1).findFirst().get();
```

6. Find First String Starting with 'A'

Input

```
List names = Arrays.asList("Bob","Alex","Anna");
```

Solution

```
names.stream().filter(s -> s.startsWith("A")).findFirst().orElse("Not Found");
```

7. Group Words by Length

Input

```
List words = Arrays.asList("Java","API","Spring","Node");
```

Solution

```
words.stream().collect(Collectors.groupingBy(String::length));
```

8. Count Occurrences of Words

Input

```
String sentence = "java spring java api";
```

Solution

```
Arrays.stream(sentence.split(" ")).collect(Collectors.groupingBy(w -> w, Collectors.counting()));
```

9. Merge Two Lists

Input

```
List list1 = Arrays.asList(1,2); List list2 = Arrays.asList(3,4);
```

Solution

```
Stream.concat(list1.stream(), list2.stream()).toList();
```

10. Convert List to Map

Input

```
List employees = ...;
```

Solution

```
employees.stream().collect(Collectors.toMap(Employee::getId, Employee::getName));
```